F-TAG: Forensic Temporal Attack Graphs for Evidence-Resilient Network Investigation

Ayan Gain[1]

Mridul Sankar Barik[1]

Jadavpur University, Kolkata, India

ayan.gain2010@gmail.com, mridulsankar.barik@jadavpuruniversity.in

October 23, 2026

Abstract

Network forensics must reconstruct multi-step attacks from evidence that is incomplete, lost to routine logging failure, or deliberately tampered with. Evidence graphs stall when records are missing, and static attack graphs ignore time-varying configurations, admitting physically impossible reconstructions. We ask whether temporally evolving networks can act as strict structural constraints on retrospective analysis. We formalize Forensic Temporal Attack Graphs (F-TAG), which enforce chronological consistency during traversal, and derive the Forensic Window of Compromise (FWoC), which bounds intrusion timelines by intersecting surviving evidence with the topological attack corridor (TAC). FWoC is provably sound under contiguous execution, and we characterize its breakdown under adversarial dormancy. To distinguish deliberate log wiping from routine loss, we formulate a Bayesian Anti-Forensics Confidence (AF_{conf}) metric from host privilege metadata. On DARPA LLDOS benchmarks and synthetic enterprise networks, F-TAG detects structural falsifications (patched or expired edges) that static attack graphs miss, while trading away adjacent-window swaps

at its operational tolerance. FWoC narrows a 13-window corridor to 8.4 windows from a single surviving alert. AF_{conf} reaches an AUC of 0.80 when wiping targets privileged hosts and 0.56 as that bias weakens (0.5 if privilege-blind), so binary privilege metadata discriminates only as far as attackers target privileged hosts.

Keywords: Network Forensics, Temporal Attack Graphs, Anti-Forensics, Bayesian Analysis

1 Introduction

Imagine an investigator responding to a breach whose only trace is an IDS alert on a database while upstream web logs are lost. Did the attacker exploit a vulnerability patched last week, or an active flaw? Was the log deliberately wiped? Without answers, investigations stall [1]. Two tangled challenges emerge. Surviving evidence may be missing or falsified, and candidate attack sequences must be validated against how the network physically evolved over time.

Existing tools solve one problem or the other, never both. Evidence graphs [2] reconstruct causal chains directly from logs and alerts. They work well when evidence is complete, but the chain simply breaks wherever a log is wiped. Conversely, static attack graphs [3, 4] are often used to bridge these gaps by correlating alerts against known vulnerability paths [5, 6]. While static attack graphs successfully map every path the network has ever permitted, they collapse that history into a single timeless snapshot, offering no notion of when each path was actually usable.

Why a static attack graph is not enough. A static attack graph combines assets and vulnerabilities from a single network snapshot. It evaluates reachability, whereas forensics must evaluate time and order.

For example, if a static attack graph built on Monday ignores a Tuesday patch, it accepts impossible Wednesday alerts through closed routes. Across multi-step attacks, it can stitch a Week 1 exploit to a Week 3 alert via a server patched in Week 2. The result is a "Frankenstein

path"[1], which is an attack sequence physically impossible in chronological order despite each step having once been permitted.

This lack of temporal enforcement causes three forensic failures. **Temporal feasibility errors** occur when an edge did not exist when required by the preceding hop. **Topology transition errors** occur when vulnerabilities disappear mid-attack. **Evidence stitching errors** occur when individual alerts match static edges, but no chronological sequence can accommodate all alerts together. In all cases, the graph fails to verify if a multi-step sequence was ever physically possible.

Temporal attack graphs (TAGs) [7] resolve the first two failure modes structurally, by requiring that the time window in which each hop is used never precede the time window used by the hop before it. This is the key difference from simply stamping timestamps onto static edges. A TAG enforces the ordering constraint jointly across the entire attack path during traversal, not edge by edge after the fact, so a Frankenstein path is never assembled in the first place. What a TAG alone cannot resolve is the third failure mode, because it lacks mechanisms to inject observed evidence against that structure or reason over gaps where evidence is missing. Forensic investigation requires precisely that capability, namely injecting retrospective evidence as a structural constraint over time-aware networks to draw sound conclusions under incomplete or contradictory logs.

F-TAG closes this gap. The relevant forensic object is not a static path, but a temporally feasible attack sequence constrained by surviving evidence. Rather than building attack graphs bottom-up from fragile logs, F-TAG enumerates temporally feasible paths from network topology alone and constrains them retrospectively with observed alerts, as illustrated in Figure 1. This rejects temporally impossible evidence outright while bounding unobserved intrusion spans through the Forensic Window of Compromise.

This paper makes the following contributions:

1. Formalizing temporal forensic reconstruction by injecting observed evidence (IDS alerts

[1]An attack sequence assembled from attack steps that were individually valid but at non-overlapping points in time, making the combined path physically impossible.

and system logs) as strict structural constraints upon time-aware network topologies.

2. Deriving the Forensic Window of Compromise (FWoC) to establish conditional soundness guarantees that bound unobserved attacker actions when evidence is missing.

3. Demonstrating temporal contradiction detection to natively identify and filter false-positive alerts and falsified evidence where methods that rely on static attack graphs fail.

4. Introducing Anti-Forensics Confidence (AF_{conf}), a Bayesian metric to quantify the likelihood of deliberate log-wiping versus routine evidence loss.

5. Validating timeline containment, chokepoint ranking invariance, and falsification filtering across DARPA LLDOS benchmarks [8, 9] and synthetic enterprise networks up to 100 hosts.

2 Related Work

2.1 Attack Graphs and Alert Correlation

Attack graphs proactively map vulnerability chains [10, 4]. Formally, a static attack graph $G = (E \cup C, R_r \cup R_i)$ consists of exploits (E), conditions (C), prerequisites (R_r), and implications (R_i) in MulVAL [11, 12]. Alert correlation matches IDS alerts against these relationships [5, 6, 13]. Because G lacks time, it cannot model patches, configuration changes, or causal order, leading correlation engines to construct backward-time causal chains, which is the core limitation motivating F-TAG.

2.2 Evidence Graphs and Network Forensics

Evidence graphs [2, 14] structure observed digital evidence into graphs for investigative traversal, but stall when logs are missing. Approaches combining them with attack graphs

Table 1: Comparison of F-TAG with existing forensic and attack graph approaches (self-assessed).

Capability	**Static AG** [3]	**Evidence Graph** [2]	**TI-HARM** [22]	**TAG** [7]	**F-TAG** (Ours)
Temporal topology	×	×	✓	✓	✓
Causal ordering	×	Partial	×	✓	✓
Evidence injection	×	✓	×	×	✓
Missing evidence bounds	×	×	×	×	✓
Anti-forensic detection	×	×	×	×	✓
False-positive filtering	×	×	×	Partial	✓

typically use probabilistic inference (Bayesian networks, Markov models) or topological causality to estimate missing events or detect inconsistencies [15, 16, 17], though static representations lack time-varying topological boundaries. At the host level, provenance graphs (e.g., HOLMES [18]) reconstruct intra-host execution flows across system calls, whereas F-TAG complements them by enforcing macro-network temporal topological boundaries across multi-host network evolutions.

2.3 Temporal and Dynamic Network Security

Dynamic assessment methods recognize that temporal constraints fundamentally change path structures [19, 20], motivating temporal risk variance analysis [21], dynamic attack surface evaluation [22], and Temporal Attack Graph (TAG) formalisms [7].

TAG formalisms [7] represent temporal topology as discrete snapshot sequences to assess future reachability, but lack mechanisms for retrospective forensic evidence. F-TAG bridges this gap by transforming temporal networks into an evidence-constrained forensic model. Table 1 compares F-TAG to existing approaches across six core capabilities.

3 Foundational Theory

To model enterprise network forensics without state-space explosion or causal ambiguity, F-TAG formalizes how time-varying physical networks constrain observed digital evidence.

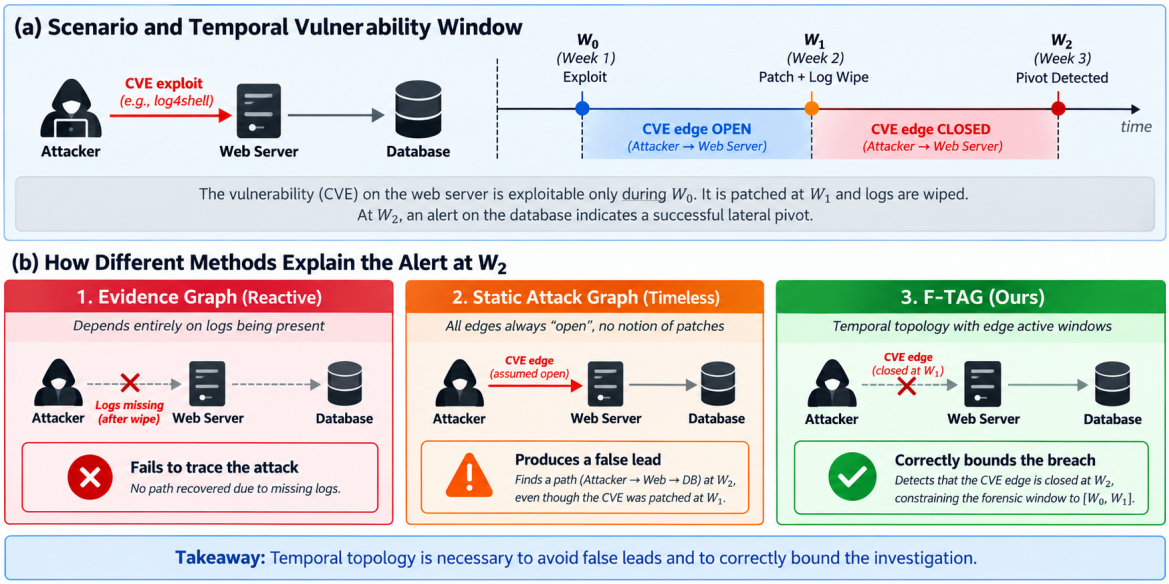


Figure 1: Motivating example. Incident timeline and comparative forensic reconstruction across evidence graphs, static attack graphs, and F-TAG under log wiping and dynamic patching.

3.1 Definition 1: Forensic Temporal Attack Graph (F-TAG)

F-TAG executes a reconstruction phase to collapse the discrete snapshots of a TAG into a unified structure where each edge e carries a set of active time windows $\mathcal{W}(e) \subseteq \{0, 1, \ldots, \tau - 1\}$, where τ denotes the total number of discrete time windows spanning the observation period (e.g., $\tau = 4$ for a four-week investigation). Throughout this paper, F-TAG operates at the host level, i.e intermediate logical deduction states are collapsed into direct host-to-host transitions, so each vertex in V_{all} corresponds to a physical network host. This representation captures the physical reality that vulnerability exploitation paths between hosts appear and disappear over time window as configurations change, patches are applied, and firewall rules are modified, without redundant duplication of shared topology across time windows.

Definition 1 (Forensic Temporal Attack Graph). Formally, Forensic Temporal Attack Graph (F-TAG) is defined as:

$$\mathcal{G}^{a}_{unified} = (V_{all},\ E_{all},\ \mathcal{W}) \tag{1}$$

where $V_{all} = \bigcup_{t=0}^{\tau-1} V_t$ is the cumulative set of all vertices, $E_{all} = \bigcup_{t=0}^{\tau-1} E_t$ is the cumulative set

of all directed attack edges, and $\mathcal{W} : E_{all} \to 2^{\{0,1,\ldots,\tau-1\}}$ maps each edge to the set of discrete time windows in which the vulnerability it represents was present and exploitable (i.e., not yet patched or revoked). The notation $2^{\{0,1,\ldots,\tau-1\}}$ denotes the power set of all time indices, meaning an edge may be active in any subset of time windows, including non-contiguous ones such as $\{0, 3\}$.

Physical Walkthrough Example. Consider an investigation spanning four discrete weeks (denoted w_0 through w_3) with time indices $\{0, 1, 2, 3\}$. Suppose directed edge e represents the exploitation of a critical web vulnerability.

1. In Week 0 (w_0), the server is unpatched. An attacker can exploit the vulnerability, so Week 0 is active, with $0 \in \mathcal{W}(e)$.

2. In Week 1 (w_1), administrators apply a vendor patch, closing the vulnerability, yielding $1 \notin \mathcal{W}(e)$.

3. In Week 2 (w_2), the system remains patched, meaning $2 \notin \mathcal{W}(e)$.

4. In Week 3 (w_3), an IT rollback accidentally restores an unpatched disk image, reopening the flaw, so $3 \in \mathcal{W}(e)$.

Under F-TAG, the edge lifespan is simply $\mathcal{W}(e) = \{0, 3\}$, meaning traversal was physically possible only during Week 0 or Week 3. An IDS alert claiming exploitation during Week 1 or Week 2 is an immediate structural contradiction. Encoding active intervals directly on edges captures the full temporal topology compactly without storing per-window graph snapshots.

3.2 Definition 2: Forensic Temporal Attack Path

Definition 2. A Forensic Temporal Attack Path p_{ij}^{f} from host i to host j is a sequence of hops $\langle (e_0, \sigma_0), (e_1, \sigma_1), \ldots, (e_{\eta-1}, \sigma_{\eta-1}) \rangle$. Here $k \in \{0, \ldots, \eta - 1\}$ is the hop index within the attack path, $\sigma_k \in \{0, 1, \ldots, \tau - 1\}$ is an integer that maps hop k to a time window index

in the F-TAG, and w_{σ_k} is the F-TAG network snapshot at that index. Concretely, $\sigma_k = j$ means hop k was executed during time window w_j of the F-TAG, i.e., the edge e_k must have been active in the network snapshot w_j. This sequence must satisfy three strict constraints:

1. **Existence Constraint:** Each edge e_k must be traversed in a time window where it was actually active, i.e., $\sigma_k \in \mathcal{W}(e_k)$. This ensures the exploitation path physically existed at the time it was used.

2. **Chronological Constraint:** The time window sequence is strictly non-decreasing: $\sigma_0 \leq \sigma_1 \leq \cdots \leq \sigma_{\eta-1}$, ensuring the attacker cannot travel backward in time.

3. **Evidence Constraint:** If an IDS alert or log entry was observed at time window w_t, then w_t must fall within the time span $[\sigma_0, \sigma_{\eta-1}]$ of the path. If no overlap exists between the observed evidence timestamps and the path's time span, the path is rejected as a structural contradiction.

Because each edge carries its active window set $\mathcal{W}(e)$ in a unified structure, these three constraints are evaluated jointly during traversal, preventing backward-time loops and rejecting historically impossible narratives to ensure every path is physically realizable.

3.3 Definition 3: Temporal Attack Corridor (TAC)

Definition 3 (Temporal Attack Corridor). For a structural candidate attack path $p = \langle e_0, e_1, \ldots, e_{H-1} \rangle$ between host i and host j, let $\Sigma(p) = \{\sigma \in \prod_{k=0}^{H-1} \mathcal{W}(e_k) : \sigma_0 \leq \sigma_1 \leq \cdots \leq \sigma_{H-1}\}$ denote the set of all temporally feasible execution sequences. The Temporal Attack Corridor (TAC) is the bounding span of discrete time windows across all feasible temporal executions:

$$\mathrm{TAC}(p) = \left[\min_{\sigma \in \Sigma(p)} \sigma_0, \quad \max_{\sigma \in \Sigma(p)} \sigma_{H-1} \right] \tag{2}$$

Because $\mathrm{TAC}(p)$ is derived exclusively from the network's topological and configuration history $\{\mathcal{W}(e_k)\}$, an investigator computes it directly from the proactive F-TAG without

knowledge of actual intrusion times. For any realized attack execution $\sigma^* \in \Sigma(p)$, the true compromise window $[\sigma_0^*, \sigma_{H-1}^*]$ is structurally guaranteed to satisfy $\sigma_0^* \geq \min_{\sigma \in \Sigma(p)} \sigma_0$ and $\sigma_{H-1}^* \leq \max_{\sigma \in \Sigma(p)} \sigma_{H-1}$, ensuring $[\sigma_0^*, \sigma_{H-1}^*] \subseteq \text{TAC}(p)$ by topological validity.

3.4 Path Enumeration Algorithm

Given a suspect entry host i (such as an external web server) and a target asset j (such as an internal database), a forensic investigator must discover all structurally feasible attack paths between them. Algorithm 1 solves this by executing a time-constrained depth-first search over the unified graph $\mathcal{G}^a_{unified}$.

Computational Intractability of Static Filtering. Generating paths on a static attack graph and filtering Frankenstein paths after the fact is computationally intractable, as simple paths scale with $O(k_{max}^{H_{max}})$, where $k_{max} = \max_v d_{out}(v)$ is the maximum node out-degree and H_{max} is the search depth limit. By enforcing $\sigma \geq \sigma_{min}$ on the fly during traversal, Algorithm 1 prunes temporally invalid branches, reducing the effective branching factor to $k_{eff} \ll k_{max}$. In structured enterprise architectures, cross-tier firewalls bound lateral movement ($H \leq 5$), so setting $H_{max} \geq 5$ (evaluated up to $H_{max} = 15$) ensures no feasible path is missed while enforcing a finite search bound.

Guaranteed Completeness. Because pruning discards only backward-time transitions ($\sigma < \sigma_{min}$), no chronologically feasible path is skipped. Algorithm 1 is mathematically guaranteed to recover every valid temporal path of length $\leq H_{max}$ between host i and host j.

4 The F-TAG Methodology

The F-TAG methodology integrates physical network evolution with observed digital evidence (such as IDS alerts, firewall logs, and authentication records) to reconstruct attack

Algorithm 1 Temporal Path Enumeration (Q_{ij}^{a})

Require: F-TAG $\mathcal{G}^a$, source node i, target node j, max hops H_{max}
Ensure: Set of all valid temporal attack paths $\mathcal{P}_{ij}$

$\mathcal{P}_{ij} \leftarrow \emptyset$
Stack $\leftarrow \{(i, 0, \langle\rangle)\}$ {(node, min_window, path)}
while Stack $\neq \emptyset$ **do**
  $(n_{curr}, \sigma_{min}, \text{path}) \leftarrow$ Stack.pop()
  if $n_{curr} = j$ **and** $|\text{path}| > 0$ **then**
    $\mathcal{P}_{ij} \leftarrow \mathcal{P}_{ij} \cup \{\text{TemporalPath(path)}\}$
    continue
  end if
  if $|\text{path}| \geq H_{max}$ **then**
    continue
  end if
  for all edges e from n_{curr} **do**
    for all $\sigma \in \mathcal{W}(e)$ where $\sigma \geq \sigma_{min}$ **do**
      Stack.push$(e.dst, \sigma, \text{path} \cup \{(e, \sigma)\})$
    end for
  end for
end while
return $\mathcal{P}_{ij}$

sequences when evidence is incomplete or contradictory.

4.1 Path-Level Forensic Bounds

When reconstructing an incident, relying solely on raw log timestamps artificially narrows the perceived timeline. Missing or wiped logs at the edges of an attack make the intrusion appear to have started later and ended earlier than it truly did. The **Forensic Window of Compromise (FWoC)** establishes the true incident timeline by intersecting what surviving evidence directly supports with what the physical network topology permits.

For an attack path p with H hops and N observed evidence timestamps $\mathcal{E} = \{t_1, \ldots, t_N\}$ mapped to discrete windows, FWoC bounds the intrusion's true start and termination by intersecting surviving evidence with the Temporal Attack Corridor (TAC):

$$\text{FWoC}(p) = \text{TAC}(p) \ \cap \ [\min(\mathcal{E}) - \delta - \delta_s, \ \ \max(\mathcal{E}) + \delta + \delta_s] \tag{3}$$

Here, $\text{TAC}(p) = [\sigma_0(p), \sigma_{\eta-1}(p)]$ defines the structural corridor, while $[\min(\mathcal{E}) - \delta - \delta_s, \max(\mathcal{E}) + \delta + \delta_s]$ expands surviving evidence timestamps by tolerance δ (clock skew and latency) and

topology correction δ_s. Their intersection retains only time windows that are both evidentially supported and topologically traversable.

The raw evidence span $[\min(\mathcal{E}), \max(\mathcal{E})]$ captures the earliest and latest timestamps recorded in surviving logs or alerts. To absorb network transit latency, clock synchronization errors between hosts, and the precision lost when mapping exact log timestamps into discrete time windows, the tolerance parameter δ (typically 0 or 1 time window) provides a small clock-skew margin.

The topology-aware correction term $\delta_s = \max(0, H - N)$ expands the raw evidence bounds outward. This accounts for the case where up to $H - N$ unobserved contiguous actions occurred entirely before the first recorded alert or entirely after the last one. When evidence is complete ($N \geq H$), $\delta_s = 0$ and the bound tightens purely around the observed timestamps.

The TAC intersection enforces a hard boundary, ensuring that an attacker cannot exploit a vulnerability before it existed or after it was patched. For example, if $H = 5$ and evidence survives for $N = 2$ hops, $\delta_s = 3$ expands evidence bounds outward by 3 windows, while the TAC clamps the span to physically active windows.

Proposition 1 (FWoC Soundness) *Let $p = (e_0, \ldots, e_{H-1})$ be a temporal attack path with true execution time windows $\sigma_0 \leq \sigma_1 \leq \cdots \leq \sigma_{H-1}$, and let $\mathcal{E} = \{t_1, \ldots, t_N\}$ be the set of surviving evidence timestamps. Under the contiguous-execution assumption (Assumption A):*

1. ***Contiguous Inter-Hop Cadence:*** *Each successive hop starts within at most one time window of the preceding hop, i.e. $\sigma_{k+1} - \sigma_k \leq 1$ for all $k \in \{0, \ldots, H-2\}$ (no inter-hop dormancy exceeding one window).*

2. ***Distinct Evidence Mapping:*** *Each of the N observed evidence timestamps corresponds to a distinct hop on p, i.e. $t_i = \sigma_{k_i}$ with indices $0 \leq k_1 < k_2 < \cdots < k_N \leq H-1$.*

the true compromise window $[\sigma_0, \sigma_{H-1}]$ is contained in $FWoC(p)$ whenever clock skew margin $\delta \geq 0$ and $\delta_s = \max(0, H - N)$.

Proof sketch: Under condition (2), exactly N distinct hops are observed, leaving at most $H-N$ unobserved hops across the path. The unobserved hops preceding the earliest observed hop k_1 number at most $k_1 \leq H - N$, and those following the latest hop k_N number at most $H-1-k_N \leq H-N$. By condition (1), each unobserved hop advances execution by at most one window. Thus, the true start window satisfies $\sigma_0 \geq \sigma_{k_1} - k_1 \geq \min(\mathcal{E}) - (H - N)$, and true termination satisfies $\sigma_{H-1} \leq \sigma_{k_N} + (H - 1 - k_N) \leq \max(\mathcal{E}) + (H - N)$. The expanded evidence interval $[\min(\mathcal{E})-\delta_s, \max(\mathcal{E})+\delta_s]$ strictly contains $[\sigma_0, \sigma_{H-1}]$. Furthermore, because the realized execution σ^* belongs to $\Sigma(p)$, Definition 3 guarantees $[\sigma_0, \sigma_{H-1}] \subseteq \text{TAC}(p)$ by topological validity. Their intersection $\text{FWoC}(p)$ therefore contains $[\sigma_0, \sigma_{H-1}]$. When condition (1) is violated (dormancy gaps > 1 window), δ_s under-expands and containment may fail, demonstrated in Table 7. □

Structural Contradiction Detection. When the intersection in Equation 3 yields an empty set ($\text{FWoC}(p) = \emptyset$), the evidence timestamps are physically incompatible with the topological attack corridor under tolerance margin δ. This contradiction flags false-positive alerts, falsified timestamps, or invalid path hypotheses. In practice, operational precision depends on calibrating δ against background timing noise. Operating at strict equality ($\delta = 0$) flags unmodeled ± 1-window clock skew on serial paths as false alarms (precision 0.75), whereas setting $\delta \geq 1$ absorbs benign transit jitter to achieve 1.00 precision, with structural violations detected by construction in Table 3.

4.2 Topological Triage & Scoping

While path metrics evaluate candidate routes, incidents across enterprise networks require prioritizing investigation resources. F-TAG aggregates surviving paths across the topology to compute structural triage metrics.

To pinpoint mandatory forensic chokepoints (hosts the adversary was forced to traverse regardless of the specific route), the temporal chokepoint score $\chi(h)$ measures the fraction

of surviving attack paths traversing host h:

$$\chi(h) = \frac{|\{p \in \mathcal{P}_{surviving} : h \in V(p)\}|}{|\mathcal{P}_{surviving}|} \tag{4}$$

where $\mathcal{P}_{surviving}$ denotes the set of candidate attack paths that remain valid after applying temporal and evidence constraints, and $V(p)$ is the set of hosts visited along path p. A score of $\chi(h) = 1.0$ confirms that host h lies on every structurally plausible attack path.

To scope log retrieval, $T_{min}(j_{tgt})$ identifies the earliest time window an attacker could have first breached the network on any path reaching target asset j_{tgt}. Derived purely from topology, T_{min} remains invariant under log deletion:

$$T_{min}(j_{tgt}) = \min_{p \in \mathcal{P}_{\to j_{tgt}}} \sigma_0(p) \tag{5}$$

where $\mathcal{P}_{\to j_{tgt}}$ is the set of topologically feasible backward-reachable attack paths terminating at a downstream victim asset j_{tgt} (such as a database exfiltration event), and $\sigma_0(p)$ is the initial entry execution time window of path p.

4.3 Anti-Forensics Confidence (AF_{conf})

When an attack path contains an unobserved hop, investigators must distinguish routine logging failure (log rotation, packet drops) from deliberate anti-forensic wiping. The Anti-Forensics Confidence metric (AF_{conf}) estimates the posterior probability of deliberate log-wiping given an evidence gap G at node n with acquired privilege level x:

$$AF_{conf}(G) = P(H_1 \mid x) = \frac{P(x \mid H_1)\, P(H_1)}{P(x \mid H_1)\, P(H_1) + P(x \mid H_0)\, P(H_0)} \tag{6}$$

where:

- H_1 is the hypothesis that the evidence was **deliberately wiped** by an attacker.

- H_0 is the hypothesis that the evidence was lost through **benign causes** (routine maintenance, logging failure).
- $P(H_1)$ and $P(H_0) = 1 - P(H_1)$ are prior probabilities reflecting baseline operational suspicion.
- $x \in \{\text{root}, \text{user}\}$ is the privilege state the attacker acquired on node n (where "root" indicates full administrative access, and "user" indicates restricted, standard access).

The likelihood terms in Equation 6 are grounded directly in operating system access-control principles. In modern operating systems, unprivileged users cannot modify or erase security event logs (such as Windows Security Event Logs, or endpoint detection telemetry), tampering with audit records strictly requires administrative or root privileges. Consequently, under the deliberate wiping hypothesis (H_1), the adversary is predominantly expected to have acquired root access:

$$P(\text{root} \mid H_1) = 0.75, \quad P(\text{user} \mid H_1) = 0.25 \tag{7}$$

These representative heuristic values reflect that log tampering predominantly requires administrative privileges (0.75), while retaining a non-zero probability (0.25) for edge cases such as user-space log deletion or lateral movement before privilege escalation. Grounding likelihood in access permissions provides robust discrimination without relying on brittle anti-forensic tool signatures that adversaries evade.

Conversely, benign logging failures (e.g., buffer overflows or network packet drops) happen indiscriminately across infrastructure regardless of host privilege. Under H_0, the probability of observing a missing record at a root-privileged node simply equals the empirical proportion of root nodes across the generated F-TAG:

$$P(\text{root} \mid H_0) = P_{root,H_0} = \frac{|\{n \in V_{all} : \text{priv}(n) = \text{root}\}|}{|V_{all}|} \tag{8}$$

F-TAG estimates P_{root,H_0} from the topology, which adapts AF_{conf} to an environment's privilege mix. This assumes benign loss is spread uniformly over hosts. If it concentrates on path nodes, whose root share can differ from the network-wide share, the score is miscalibrated in Section 5.4. This formulation yields two analytical properties that govern the behavior of AF_{conf}.

Lemma 1 (Monotonicity under Equal Base Rates) *If $P(x_{root} \mid H_0) = P(x_{user} \mid H_0)$ (i.e. the base rate of missing evidence is the same across privilege levels), then AF_{conf} is strictly higher at root nodes than at user nodes.*

Proof: Let $c = P(x_{root} \mid H_0) = P(x_{user} \mid H_0)$. By our domain grounding, $P(x_{root} \mid H_1) = 0.75 > 0.25 = P(x_{user} \mid H_1)$. The posterior $AF_{conf}(x) = \frac{P(x|H_1)P(H_1)}{P(x|H_1)P(H_1)+c\,P(H_0)}$ is strictly increasing in $P(x \mid H_1)$ for fixed $c > 0$. Hence $AF_{conf}(x_{root}) > AF_{conf}(x_{user})$. Concretely, at a 10% prior with $c = 0.5$ (half the nodes are root, i.e. truly uniform), this gives $\approx 14\%$ at a root node versus $\approx 5\%$ at a user node. □

Note: When $P(x_{root} \mid H_0) \neq P(x_{user} \mid H_0)$, the ordering depends on the specific base rates. The lemma's equal-base-rate condition is the sanity case, while the general formula in Equation 6 handles arbitrary base rates directly.

Boundary Condition (Root Saturation Inversion): In networks saturated with root-privileged nodes ($P(x \mid H_0) \to 1$), the base rate for non-root nodes approaches zero ($P(x_{user} \mid H_0) \to 0$). Here, missing evidence at a rare non-root node produces an artificially high AF_{conf} score, inverting the expected ordering.

Lemma 2 (Root-Node Posterior Limit under Saturation) *As the network becomes saturated with root-level nodes ($P(x \mid H_0) \to 1$), the AF_{conf} score for root nodes drops strictly below the prior $P(H_1)$.*

Proof: Let $P(x \mid H_0) \to 1$. With operational setting $P(x \mid H_1) = \alpha = 0.75$, the limit is $\frac{0.75P(H_1)}{0.75P(H_1)+P(H_0)}$. Since $P(H_1) < 1$ and $P(H_0) > 0$, this is strictly less than the prior $P(H_1)$.

The limit only approaches $P(H_1)$ if we assume deliberate wiping *always* occurs at root nodes ($\alpha \to 1$). $\square$

Sanity Properties of AF_{conf}. Together, Lemma 1 and Lemma 2 formally characterize AF_{conf} under extreme baselines. Lemma 1 establishes the standard sanity property that root nodes are strictly prioritized when operational loss is privilege-blind. Lemma 2, along with the saturation inversion boundary condition, exposes the metric's limit under pathological baselines. In root-saturated networks, missing evidence at a rare non-root node scores near 1 ($P(x_{user} \mid H_0) \to 0$), causing false accusations of deliberate wiping to concentrate heavily on rare user-level nodes while deflating suspiciousness on the root nodes adversaries actually target.

5 Evaluation

To evaluate F-TAG, its performance is assessed across two complementary environments.

- **Historically Verified Intrusions (DARPA LLDOS 1.0 and 2.0.2)** [8, 9]: Provides packet-level ground truth to test timeline reconstruction, FWoC bounds under evidence degradation, topological triage stability, and adversarial falsification detection.
- **Scalable Enterprise Networks (MulVAL)** [11]: Generates multi-tier enterprise networks to validate temporal Datalog mapping, Bayesian AF_{conf} anti-forensics calibration, and computational scalability up to 100 hosts.

5.1 Evaluation Environments and Setup

DARPA LLDOS Benchmarks. We mapped the verified capability paths from both LLDOS scenarios into F-TAGs, using the real, historical packet captures and IDS alerts from the dataset as our evidence base. LLDOS 1.0 represents an attacker executing a multi-stage

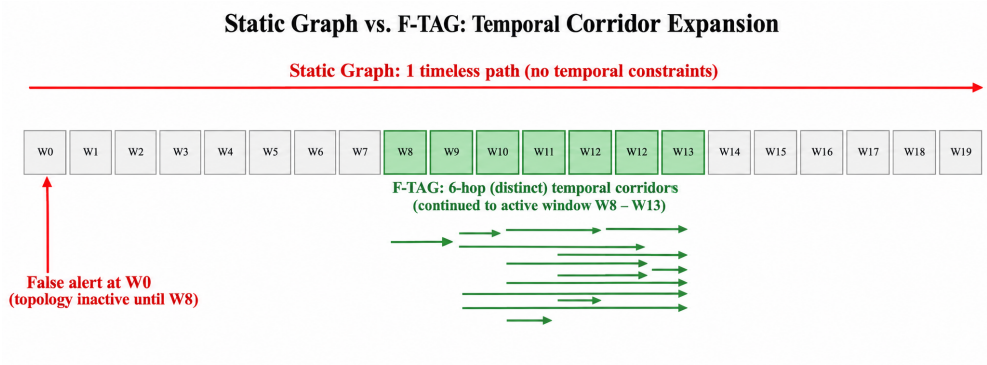


Figure 2: Static attack graph (single timeless path) versus F-TAG (252 valid temporal execution sequences) on a 5-hop chain topology across $\tau = 20$ time windows (w_8, w_{13} active).

distributed denial-of-service (DDoS) attack across multiple hosts (`mill`, `pascal`, `locke`), exhibiting topological path redundancy. In contrast, LLDOS 2.0.2 models a stealthy, serial intrusion executing without topological redundancy.

Enterprise Topology Generation. We procedurally generated synthetic multi-tier enterprise network topologies and used MulVAL [11] to generate corresponding attack graphs across discrete time windows, modeling time-varying vulnerability lifespans and configuration updates. Attack paths and evidence logs across generated networks were evaluated via a Monte Carlo injection pipeline to validate our metrics at scale.

Static vs. Temporal Attack Feasibility. To illustrate temporal constraints, we evaluated a 5-hop path (6 hosts) across $\tau = 20$ windows where vulnerabilities coexisted only during $[w_8, w_{13}]$. While static attack graphs collapse this into a single timeless path, F-TAG identifies the 252 valid temporal paths satisfying $\sigma_0 \leq \cdots \leq \sigma_4$ within $[8, 13]$, as illustrated in Figure 2. An alert outside this active window (e.g., false alarm at w_0) yields FWoC $= \emptyset$, natively filtering impossible alerts that static attack graphs accept.

5.2 Timeline Reconstruction and FWoC Bounds

Table 2 checks Proposition 1 on both LLDOS datasets under evidence degradation (100 Monte Carlo trials at 25%, 50% and 100% completeness). Containment is guaranteed under

Table 2: Consistency check of Proposition 1 under evidence degradation for LLDOS 1.0 and 2.0.2 (100 Monte Carlo trials each). Soundness reports 95% Wilson score confidence intervals. Tightness represents the mean FWoC width as a percentage of observation duration τ. Ambiguity $k = |\mathcal{P}_{surviving}|$ denotes surviving candidate paths. *For LLDOS, overlap and containment soundness coincide due to tight true execution spans.

Dataset	Completeness	Soundness (Containment)*	Tightness	Ambiguity (k)	$\mathbf{T_{min}}$
LLDOS 1.0	100%	100.0% (96.3% to 100.0%)	100.0%	1.00	2.0
	50%	100.0% (96.3% to 100.0%)	100.0%	1.00	2.0
	25%	100.0% (96.3% to 100.0%)	100.0%	1.00	2.0
LLDOS 2.0.2	100%	100.0% (96.3% to 100.0%)	100.0%	1.00	0.0
	50%	100.0% (96.3% to 100.0%)	100.0%	1.00	0.0
	25%	100.0% (96.3% to 100.0%)	100.0%	1.00	0.0

Assumption A, so 100% here confirms the implementation rather than measuring performance. FWoC's empirical contribution is narrowing, which LLDOS cannot show because $\tau = 5 \leq H$, so we measure it on a synthetic chain reported in Table 7.

FWoC Narrowing and Dormancy. We embed a 5-hop MulVAL path in $\tau = 50$ windows. Blocks A and B are exact enumerations over observed-hop subsets, and only Block C is Monte Carlo (30 seeds $\times$ 100 trials). With vulnerabilities present only in windows 8–13, TAC $= [8, 13]$ equals the intrusion span, so TAC alone gives the whole 88% reduction $(1 - 6/50)$ and evidence adds nothing. Where vulnerabilities outlive the intrusion (TAC $= [6, 18]$, breach at $[8, 12]$), evidence narrows the bound below the corridor, reported in Table 7 (Block A), where containment is 100% for every N under Assumption A. Under a 2-window gap that violates Assumption A (Block B), containment falls to 60% at $N = 1$, and more evidence can lower it (90% at $N = 3$, 80% at $N = 4$), because each extra observed hop shrinks $\delta_s = H - N$ before the gap is pinned. Block C repeats the test with an 18-window dormancy gap ($\tau = 30$), where containment is 0% at $N = 1$. Overlap soundness stays 100% in blocks A and B.

The Chokepoint Ranking and Breach Window Anchor Survive Total Log Loss. Since T_{min} is derived from topology rather than surviving evidence, it stayed pinned to the true breach window at every completeness level ($T_{min} = 2.0$ for LLDOS 1.0, $T_{min} =$

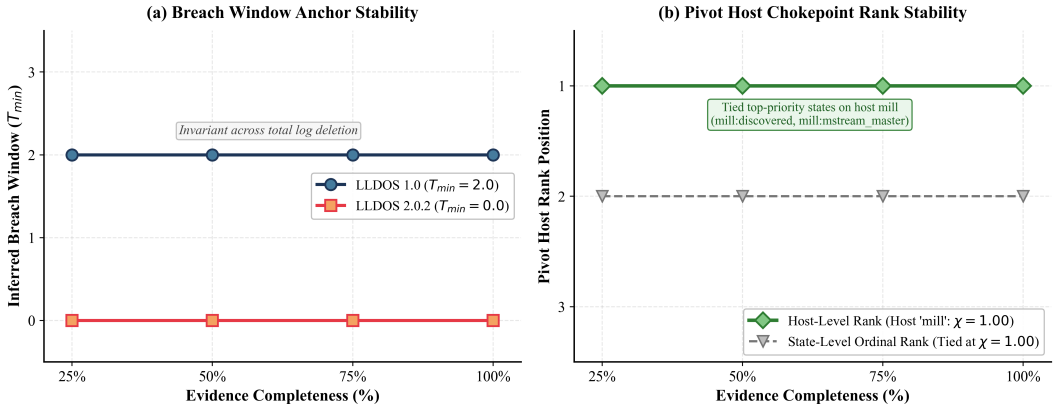


Figure 3: (a) Topological breach window anchor T_{min} stability across evidence loss. (b) Pivot host Temporal Chokepoint Score $\chi(h)$ rank stability, illustrating topological invariance under log destruction and resolving state-level ties.

0.0 for LLDOS 2.0.2), as shown in Figure 3(a), so evidence destruction cannot move it. The true pivot host (`mill`) likewise maintained top-priority ranking ($\chi = 1.0$, Rank #1 among physical hosts), as plotted in Figure 3(b), regardless of evidence loss. At the state level, all four compromise states on `mill` tie at $\chi = 1.0$, where arbitrary sorting places `mill:mstream_master` behind prerequisite `mill:discovered`, but both belong to the same host and share top rank under competition ranking.

5.3 Adversarial Falsification Detection

We evaluated F-TAG's ability to expose tampered evidence against the static attack graph baseline (Zhang et al. [16]), using identical node-edge topologies without temporal ordering constraints. We tested Monte Carlo sweeps across 30 independent seeds (50 trials per seed = 1,500 trials per scenario) across three adversarial falsification strategies, detailed in Table 3 and Figure 4.

- **Precedence Swap**: Exchanging the chronological timestamps of two sequential attack hops.
- **Vulnerability Patching**: Submitting an alert for an exploit hop after a permanent patch removed the vulnerability.

Table 3: Falsification detection under ±1-window clock skew noise headlined at operational tolerance $\delta = 1$, with an empirical tolerance and noise stress sweep on LLDOS 2.0.2 ($\tau = 5$, 30 seeds, 1,500 trials/scenario, 95% CIs). Setting $\delta = 1$ absorbs ±1 skew by construction (precision 1.00) while trading away 1-window precedence swap detection. Recall on structural violations (patching/expiration) is guaranteed by topological construction (FWoC $= \emptyset$).

Scenario / Dataset	**Zhang (Static Baseline)**		**F-TAG (Ours, $\delta = 1$)**	
	Precision	**Recall**	**Precision**	**Recall**
Headlined Performance at $\delta = 1$ (Noise Model: Skew $\in \{-1, 0, +1\}$)				
LLDOS 1.0 (Full Wipe: Precedence)	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	0.00 ± 0.00
LLDOS 1.0 (Full Wipe: Patching)	1.00 ± 0.00	0.00 ± 0.00	1.00 ± 0.00	0.81 ± 0.02[2]
LLDOS 1.0 (Full Wipe: Expiration)	1.00 ± 0.00	0.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00
LLDOS 2.0.2 (Serial: Precedence)	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	0.00 ± 0.00
LLDOS 2.0.2 (Serial: Patching)	1.00 ± 0.00	0.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00
LLDOS 2.0.2 (Serial: Expiration)	1.00 ± 0.00	0.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00
Tolerance Parameter and Noise Stress Sweep on LLDOS 2.0.2				
Configuration	**Precision**	**Rec (Preced.)**	**Rec (Patch.)**	**Rec (Expir.)**
$\delta = 0$ (Strict, skew ±1)	0.75 ± 0.01	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00
$\delta = 1$ (Operational, skew ±1)	1.00 ± 0.00	0.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00
$\delta = 1$ (Stress: skew ±2)	0.92 ± 0.01	0.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00

- **Connectivity Expiration**: Submitting an alert after a scheduled, time-bounded connectivity time window expired.

Falsification Detection: Gains, Losses, and Tolerance Calibration. Table 3 headlines performance at operational tolerance $\delta = 1$ under a ±1-window clock skew noise model, with an empirical tolerance and noise stress sweep. Across all datasets at $\delta = 1$, F-TAG achieves 1.00 precision by construction because tolerance margin $\delta = 1$ accommodates the injected ±1-window transit jitter in clean background traffic. On Vulnerability Patching and Connectivity Expiration, F-TAG achieves perfect recall (1.00) under full infrastructure wipes (0.81 on LLDOS 1.0 due to redundant lateral paths), as expected by topological construction. Because the exploit edges are structurally absent from the F-TAG during the falsified windows, no valid path can be formed (FWoC $= \emptyset$). Static baselines (Zhang et al.) lack time-varying topology, scoring 0.00 recall on both structural attacks.

Conversely, on 1-window Precedence Swaps, F-TAG recall falls to 0.00 at $\delta = 1$, while the

[2]Recall is ≈ 0.81 because revoking an interior step has a 1-in-5 chance of hitting the lateral edges into `locke/pascal`, whereas `mill` lacks this step and survives.

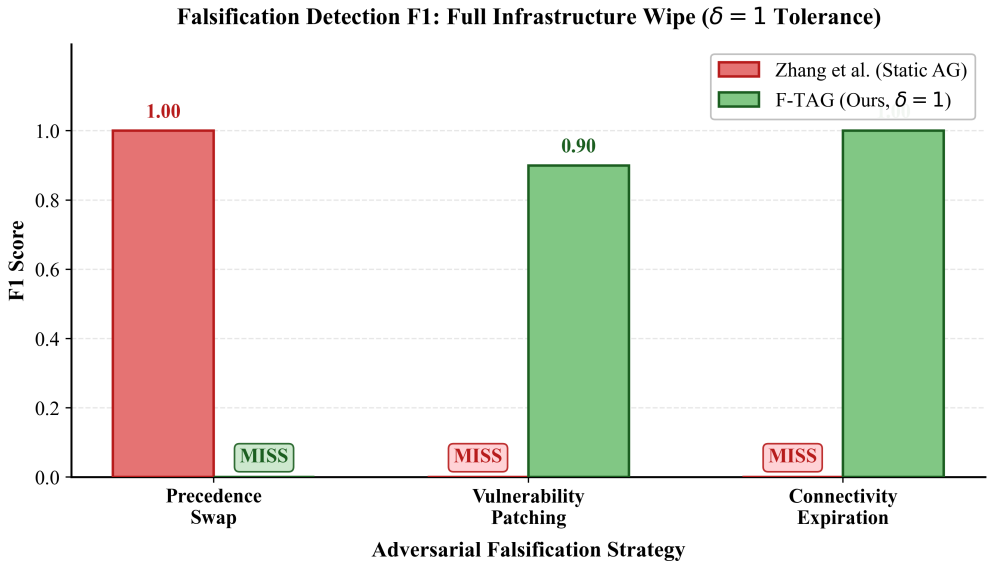


Figure 4: F1 score comparison across three falsification scenarios under Full Infrastructure Wipe at operational tolerance $\delta = 1$ (Zhang static baseline vs. F-TAG).

static baseline achieves 1.00. Setting $\delta = 1$ directly trades away precedence-swap detection because adjacent 1-window timestamp inversions fall entirely within the ± 1-window tolerance margin designed to absorb transit jitter. The strategy-decoupled sweep on LLDOS 2.0.2 in Table 3 makes this trade-off explicit. At $\delta = 0$ (strict window equality), F-TAG catches 100% of 1-window precedence swaps, but self-inflicts false alarms from unmodeled clock skew on the serial path, lowering precision to 0.75. Setting $\delta \geq 1$ eliminates false alarms (precision 1.00) while macroscopic structural violations remain caught with 1.00 recall. To demonstrate where this precision guarantee ends, Table 3 includes a stress test with ± 2-window clock skew at $\delta = 1$, where precision drops to 0.92 because timing noise exceeds the margin. Finally, Figure 4 illustrates this operational trade-off in F1 scores. Zhang achieves $F_1 = 1.00$ on Precedence Swaps but misses both structural violations ($F_1 = 0.00$), whereas F-TAG ($\delta = 1$) trades away 1-window swaps ($F_1 = 0.00$) to achieve $F_1 = 0.90$ on Patching and $F_1 = 1.00$ on Expiration.

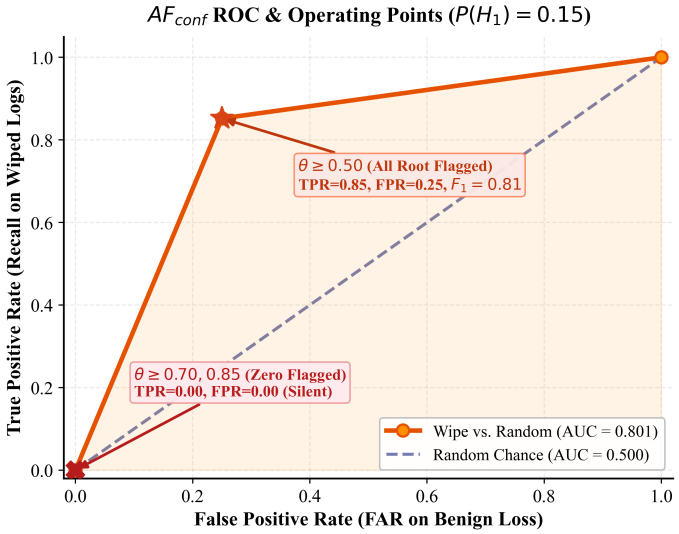


Figure 5: Receiver Operating Characteristic (ROC) curve of AF_{conf} on MulVAL ($P(H_1) = 0.15$, AUC = 0.801) annotated with the two discrete operating points from Table 4: $\theta \geq 0.50$ (all root nodes flagged) and $\theta \geq 0.70, 0.85$ (complete classification silence).

5.4 Bayesian Anti-Forensics Calibration (AF_{conf})

The central operational question is whether AF_{conf} can reliably tell deliberate log-wiping apart from routine telemetry loss, and whether that discrimination depends on how suspicious we start out (the prior $P(H_1)$).

To evaluate this, we calibrated AF_{conf} using a Monte Carlo injection pipeline across 30 independent seeds and three prior settings ($P(H_1) \in \{0.05, 0.15, 0.30\}$) on the synthetic MulVAL F-TAG (900,900 total evaluation instances across 15 candidate paths, with 30,030 samples per seed), testing three distinct evidence-loss classes:

1. **Wipe-Class (H_1)**: Deliberate log-wiping biased toward high-privilege pivot nodes (10× weight for root status) and scaled by temporal chokepoint score χ.

2. **Random-Benign (H_0)**: Uniformly distributed evidence loss across all path nodes, modeling accidental failure.

3. **Rotation-Benign (H_0)**: Periodic log loss recurring at fixed machines, modeling scheduled log rotation.

It Does Not Depend on the Prior. Lemma 3 proves that the ranking AF_{conf} induces is strictly prior-invariant. Figure 5 illustrates the resulting ROC curve (AUC = 0.801 for Wipe vs. Random loss) and maps the discrete operating points induced by binary privilege.

Lemma 3 (Prior Invariance of Ranking) *For fixed $x \in \{root, user\}$, the posterior*

$$AF_{conf}(x) = \frac{P(x \mid H_1)P(H_1)}{P(x \mid H_1)P(H_1) + P(x \mid H_0)P(H_0)}$$

is a strictly increasing function of the likelihood ratio $LR(x) = \frac{P(x|H_1)}{P(x|H_0)}$ for any $P(H_1) \in (0, 1)$. Since the set of achievable LR values (and hence the induced ranking over samples) is independent of $P(H_1)$, the ROC curve, and therefore AUC, is invariant to the choice of prior.

Proof: Write $AF_{conf}(x) = \frac{LR(x)P(H_1)}{LR(x)P(H_1)+(1-P(H_1))}$, which is strictly increasing in $LR(x)$ for any fixed $P(H_1) \in (0, 1)$ since $\frac{\partial AF_{conf}}{\partial LR} = \frac{P(H_1)(1-P(H_1))}{(\cdot)^2} > 0$. As $LR(x)$ does not depend on $P(H_1)$, the ordering of AF_{conf} across the two values of x is preserved for every $P(H_1)$, so the ROC curve (which depends only on this ordering) is unchanged. □

Proposition 2 (Privilege-Blind Loss) *If $P(x \mid H_1) = P(x \mid H_0)$ for every x, then $AF_{conf}(x) = P(H_1)$ for all x, and any ranking by AF_{conf} has AUC = 0.5.*

Proof. Here $LR(x) = 1$, so $AF_{conf}(x) = P(H_1)/(P(H_1) + 1 - P(H_1)) = P(H_1)$. The score is constant and ranks all samples equally. □

Privilege metadata therefore separates deliberate wiping from routine loss only to the extent that wiping targets privileged hosts. Table 5 measures this dependence, so AUC ≈ 0.80 should not be read as a property of the metric alone.

Operating Points and Step-Function Threshold Behavior. Under the binary privilege formulation with operational prior $P(H_1) = 0.15$, AF_{conf} evaluates to exactly two discrete score levels, namely 0.0475 for unprivileged nodes and 0.5323 for root/administrative

Table 4: AF_{conf} operating points and threshold behavior under binary privilege metadata ($P(H_1) = 0.15$, 30 independent seeds, 900,900 total evaluation samples). Thresholds $\theta \geq 0.70$ (including candidate cutoff $\theta = 0.85$) do not reflect high-confidence discrimination, but rather yield zero flagged alerts (Recall = 0.0%, FAR = 0.0%) due to complete classification silence under binary privilege's two discrete posterior values (0.0475 and 0.5323).

Threshold θ	# Flagged	Precision	Recall	F1	FAR (Random)	FAR (Rotation)
≥ 0.50	496,279	0.773	0.852	0.810	25.0%	25.4%
≥ 0.70	0	—	0.000	—	0.0%	0.0%
≥ 0.85	0	—	0.000	—	0.0%	0.0%
≥ 0.90	0	—	0.000	—	0.0%	0.0%

nodes. Because of this binary structure, threshold behavior is a step function rather than a continuous curve, as detailed in Table 4. At threshold 0.50, all root nodes are flagged (496,279 samples across 30 seeds), achieving 0.773 precision, 0.852 recall, and 0.810 F1, while incurring a false-accusation rate (FAR) of 25.0% against random benign loss (25.4% against rotation loss). For any threshold above 0.5323 (including 0.70, 0.85, and 0.90), the classifier flags exactly zero alerts. Thus, a 0.0% FAR at 0.85 would reflect classification silence, not discrimination. The scores assume $P_{root,H_0} = 11.6\%$, the root share of all hosts, whereas 25.0% of Random-Benign samples fall on root nodes. FAR at $\theta \geq 0.50$ equals that root share, more than twice the 11.6% the scores assume. In practice, operational alerting requires either calibrating the baseline prior $P(H_1)$ to shift the discrete score levels or incorporating multi-tier privilege models.

Privilege Bias Circularity and Limitations. To test whether the headline discrimination (AUC ≈ 0.801) is driven by F-TAG's structural modeling or circular dependence on the wipe generator's privilege bias, we swept the wipe generator's root-node selection weight across $1\times, 2\times, 5\times$, and $10\times$ across 30 seeds (900,900 samples total) in Table 5. At $1\times$ the wipe class still contains 36.5% root nodes, because chokepoint scaling favors pivot hosts, so AUC falls to 0.5574 with 95% CI [0.5566, 0.5583] rather than to the privilege-blind floor of 0.5 (Proposition 2). As root targeting bias increases to $2\times$, $5\times$, and $10\times$, the mean AUC rises monotonically to 0.6421, 0.7459, and 0.8007, respectively. This demonstrates that AF_{conf}'s discriminative power strictly tracks the attacker's own privilege targeting bias rather than

Table 5: Root-weight sweep in the synthetic wipe generator ($P(H_1) = 0.15$, 30 independent seeds). Mean AUC tracks the generator's privilege bias monotonically, exposing that discriminative power is driven by privilege weight rather than independent topology.

Root Weight	Mean AUC	95% CI	Root Proportion in Wipe Class
1×	0.5574	[0.5566, 0.5583]	36.5%
2×	0.6421	[0.6412, 0.6431]	53.4%
5×	0.7459	[0.7450, 0.7467]	74.2%
10×	0.8007	[0.8000, 0.8015]	85.2%

Table 6: F-TAG Empirical Scalability across 3-Tier Enterprise Networks ($\tau = 5$ windows, $H_{max} \leq 5$). Proactive MulVAL generation runs offline, while online triage measures path enumeration and metric evaluation.

Hosts	Edges	Proactive Gen (s)	Candidate Paths ($H \leq 5$)	Online Triage (s)
9	17	6.5	40	0.0001
29	189	9.8	648	0.0008
50	560	10.6	2,048	0.0027
69	1,057	14.8	3,960	0.0051
99	2,177	21.2	8,320	0.0128

an independent structural signal. We report this honestly as a limitation of binary privilege metadata. When an attacker wipes logs uniformly without privilege preference, AF_{conf} cannot separate deliberate wiping from accidental loss. Addressing this requires privilege-agnostic loss models (e.g., volumetric anomaly detection and telemetry continuity checks) in future work.

Graceful Failure without Metadata. On datasets without privilege metadata (such as raw LLDOS), AF_{conf} cannot discriminate at all and correctly reports that the AUC collapses to 0.5 (chance) rather than producing a confident-looking wrong answer.

5.5 Computational Scalability

We evaluated the computational performance of F-TAG on synthetic F-TAGs representing three-tier enterprise architectures (DMZ, Application, Database) scaled from 9 to 99 physical hosts across five temporal time windows. Intermediate logical deduction rules were collapsed into host-to-host physical transitions to reflect realistic triage overhead.

Table 6 reports computational scalability, separating proactive offline generation (6.5–

21.2 s for 99 hosts) from online reactive triage. For the 99-host, 2,177-edge network, Algorithm 1 enumerates all 8,320 candidate temporal paths in 7.5 ms and evaluates metrics in 2.8 ms (10.3–12.8 ms total). Importantly, the absence of a 22^5 path explosion at $H_{max} \geq 3$ is a structural property of the synthetic generator. It models strict DMZ $\rightarrow$ App $\rightarrow$ Database progression where inter-tier ACLs render all simple ingress-to-target paths exactly 3 hops, so setting $H_{max} = 5$ (or 15) explores the exact same 8,320 paths in 11.4 ms enumeration (14.3 ms total triage). Real enterprise networks, however, feature peer-to-peer lateral movement within subnets. To test this boundary, we injected intra-tier lateral edges within the DMZ and Application tiers. At $H_{max} = 5$, candidate paths expand moderately from 8,320 to 13,223, enumerating in 14.9 ms. In dense, unpartitioned mesh topologies with unconstrained cycles, paths scale with $O(k_{max}^{H_{max}})$, where bounding $H_{max} \leq 5$ and applying standard subnet decomposition in Section 6 is required to maintain real-time triage.

6 Discussion and Threats to Validity

By injecting evidence as strict filters over time-varying networks, F-TAG eliminates static attack graphs' structural failure modes while sustaining real-time triage ($O(|\mathcal{P}_{surviving}| \cdot H_{max})$, under 15 ms for 99 hosts). However, F-TAG operates within six boundaries. First, empirical validation relies on 2000-era DARPA LLDOS data and author-built simulators for falsifications, wipes and noise (bounded ± 1 skew in Table 3). Second, topological redundancy can absorb contradictory evidence when parallel paths exist, bounding falsification detection. Third, AF_{conf} thresholds were calibrated on synthetic graphs, so evaluating against live SIEM telemetry with verified ground truth is the next step. Fourth, the binary privilege model serves as a baseline floor, while richer hierarchies (SELinux, AD) will expand discrimination. Fifth, dense topologies benefit from subnet aggregation to sustain enterprise scalability. Finally, F-TAG treats configuration snapshots and scans as tamper-resistant ground truth. If scan data originates from compromised infrastructure, the reference frame

Table 7: FWoC width and containment across contiguous execution and dormancy gaps (MulVAL synthetic, where Blocks A and B are exact enumerations over observed-hop subsets and only Block C is Monte Carlo with 30 seeds $\times$ 100 trials).

Block	N	Width (% of τ)	Containment
A: Contiguous, $\tau = 50$, TAC=[6,18]	1	16.8%	100%
	2	17.2%	100%
	3	16.0%	100%
	4	13.2%	100%
	5	10.0%	100%
B: 2-window gap, $\tau = 50$	1	16.8%	60%
	2	18.0%	90%
	3	17.2%	90%
	4	14.8%	80%
	5	12.0%	100%
C: 18-window dormancy, $\tau = 30$	1	17.9%	0%
	2	49.7%	62%
	5	70.0%	100%

itself is compromised, representing a trust boundary common to all attack graphs.

Adversarial Dormancy Boundary. Block C of Table 7 stress-tests FWoC against a simulated dormant APT with an 18-window gap, which violates Assumption A, where containment is 0% at $N = 1$ and recovers to 100% only at $N = 5$, where $\delta_s = 0$ and every hop is observed. Analysts investigating dormancy should widen δ_s or treat FWoC as an inner estimate of the intrusion window. A replication package is available at `https://github.com/ayangain/TAG-Forensics`.

7 Conclusion

We introduced F-TAG to constrain retrospective sequence reconstruction over time-varying networks. We derived the Forensic Window of Compromise (FWoC) to bound unobserved actions, characterized its breakdown under dormancy, and formulated AF_{conf} to evaluate deliberate log wiping. Across DARPA LLDOS and enterprise networks, F-TAG eliminates timeless path assembly and detects structural falsifications that static attack graphs miss, while AF_{conf}'s discrimination tracks attacker privilege targeting (AUC 0.80 falling to 0.56).

8 Future Work

Future work will explore multi-level privilege hierarchies (e.g., SELinux and Active Directory) and telemetry continuity checks to address privilege-bias circularity. Extending F-TAG to continuous sliding windows and live SIEM telemetry represents the natural next step toward operational deployment.

References

[1] Muhammad Naeem Khan, Chris R Chatwin, and Rupert CW Young. Digital forensics and incident response: A survey. *IEEE Access*, 10:126437–126458, 2022.

[2] Wei Wang and Thomas E Daniels. A graph based approach toward network forensics analysis. *ACM Transactions on Information and System Security (TISSEC)*, 12(1):1–33, 2008.

[3] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J.M. Wing. Automated generation and analysis of attack graphs. In *Proceedings 2002 IEEE Symposium on Security and Privacy*, pages 273–284, 2002.

[4] Harjinder Singh Lallie, Kurt Debattista, and Jay Bal. A review of attack graph and attack tree visual syntax in cyber security. *Computer Science Review*, 35:100219, 2020.

[5] Peng Ning, Yun Cui, and Douglas S. Reeves. Constructing attack scenarios through correlation of intrusion alerts. In *Proceedings of the 9th ACM Conference on Computer and Communications Security*, CCS '02, page 245–254, New York, NY, USA, 2002. Association for Computing Machinery.

[6] Peng Ning and Dingbang Xu. Building attack scenarios through integration of complementary alert correlation methods. In *Proceedings of the 11th Annual Network and Distributed System Security Symposium (NDSS)*, 2004.

[7] Ayan Gain and Mridul Sankar Barik. Attack graph based security metrics for dynamic networks. In Vallipuram Muthukkumarasamy, Sithu D. Sudarsan, and Rudrapatna K. Shyamasundar, editors, *Information Systems Security*, pages 109–128, Cham, 2023. Springer Nature Switzerland.

[8] Joshua W. Haines, Laura M. Rossey, Richard P. Lippmann, and Robert K. Cunningham. Extending the DARPA off-line intrusion detection evaluations. In *Proceedings of the DARPA Information Survivability Conference and Exposition II (DISCEX '01)*, volume 1, pages 35–45. IEEE, 2001.

[9] MIT Lincoln Laboratory. 2000 DARPA intrusion detection scenario specific datasets. `https://www.ll.mit.edu/r-d/datasets/2000-darpa-intrusion-detection-scenario-specific-datasets`, 2000.

[10] Steven Noel. *A Review of Graph Approaches to Network Security Analytics*, pages 300–323. Springer International Publishing, Cham, 2018.

[11] Xinming Ou, Sudhakar Govindavajhala, and Andrew W. Appel. Mulval: A logic-based network security analyzer. In *Proceedings of the 14th Conference on USENIX Security Symposium - Volume 14*, SSYM'05, page 8, USA, 2005. USENIX Association.

[12] Massimiliano Albanese, Sushil Jajodia, and Steven Noel. Time-efficient and cost-effective network hardening using attack graphs. In *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2012)*, pages 1–12, 2012.

[13] Martin Husák, Jana Komárková, Elias Bou-Harb, and Pavel Čeleda. Survey of attack projection, prediction, and forecasting in cyber security. *IEEE Communications Surveys & Tutorials*, 21(1):640–660, 2019.

[14] Jun He and Guoyan Xu. Network forensics evidence graph vulnerability reasoning. *Journal of Communications*, 11(4):377–382, 2016.

[15] Denis Liu and Yusheng Ji. Using attack graphs in forensic examinations. In *2012 IEEE International Conference on Communications (ICC)*, pages 6719–6724. IEEE, 2012.

[16] Yuqiang Zhang, Jingsha He, and Jing Xu. Detecting falsified timestamps in evidence graph via attack graph. In *2015 8th International Symposium on Computational Intelligence and Design (ISCID)*, volume 2, pages 369–374. IEEE, 2015.

[17] Marcel Frigault, Lingyu Wang, Sushil Jajodia, and Anoop Singhal. *Measuring the Overall Network Security by Combining CVSS Scores Based on Attack Graphs and Bayesian Networks*, pages 1–23. Springer International Publishing, Cham, 2017.

[18] Sadegh M Milajerdi, Rigel Gjomemo, Birhanu Eshete, R Sekar, and VN Venkatakrishnan. HOLMES: Real-time apt detection through correlation of suspicious information flows. In *2019 IEEE Symposium on Security and Privacy (SP)*, pages 1137–1152. IEEE, 2019.

[19] Petter Holme and Jari Saramäki. Temporal networks. *Physics Reports*, 519(3):97–125, 2012. Temporal Networks.

[20] Arnaud Casteigts, Paola Flocchini, Walter Quattrociocchi, and Nicola Santoro. Time-varying graphs and dynamic networks. *CoRR*, abs/1012.0009, 2010.

[21] Malik Shahzad Kaleem Awan, Pete Burnap, and Omer Rana. Identifying cyber risk hotspots: A framework for measuring temporal variance in computer network risk. *Computers & Security*, 57:31 – 46, 2016.

[22] Simon Yusuf Enoch, Jin B. Hong, and Dong Seong Kim. Security modelling and assessment of modern networks using time independent graphical security models. *Journal of Network and Computer Applications*, 148:102448, 2019.